Discovering Customer Segments and Product Associations — a KDD Project

Author: `<your name>` · **Course:** CMPE 255 · **Repo:** MeronDT/CMPE-255-Data-Mining-process · **App:** `<deployed URL>`

Data: *Online Retail II* — UCI Machine Learning Repository (dataset 502, CC BY 4.0), donated by D. Chen. All 1,067,371 transaction lines of a UK-based online retailer of giftware (many customers are wholesalers), 1 Dec 2009 – 9 Dec 2011.

KDD (Fayyad, Piatetsky-Shapiro & Smyth, 1996) is "the nontrivial process of identifying **valid, novel, potentially useful, and ultimately understandable** patterns in data". It defines nine steps, usually grouped into five stages:

Stage	Fayyad's steps	In this notebook
0 · Domain & goals	1. Understand the application domain, prior knowledge and the end user's goals	business context, two discovery goals, evaluation plan
1 · **Selection**	2. Create a target data set	load both years, define discovery vs. validation periods
2 · **Preprocessing**	3. Data cleaning and preprocessing	cancellations, non-product codes, invalid prices, duplicates, missing customers
3 · **Transformation**	4. Data reduction and projection	customer RFM table (log, scaling, PCA); invoice × product basket matrix
4 · **Data mining**	5. Match goals to a mining method · 6. Choose algorithms · 7. Mine	K-means segmentation · FP-Growth association rules
5 · **Interpretation / Evaluation**	8. Interpret mined patterns	profile segments and rules; test them on the **next year's** data
→ Act	9. Act on the discovered knowledge	consolidated outputs + a web app for marketing staff

Headline results: four customer segments that are stable (bootstrap ARI 0.94) and predict next year's behaviour — the top 17 % of customers ("Champions") produced 64 % of revenue and 96 % of them bought again the following year, against 40 % of "Lapsed one-off buyers". 579 association rules, of which 86 % meet the full thresholds again in the following year; the most useful are *matching-design sets* (e.g. red spotty paper napkins + cups → plates, 84 % confidence). Both are delivered in a web app for marketing staff.

How to run: *Runtime → Run all* (≈ 3–6 min on free Colab; most of it is downloading and reading the 44 MB Excel file once — later runs use a cache). Every figure, table and the app are written to `artifacts/`.

⚙️ Setup & configuration

In [1]:
```
try:
    import mlxtend, openpyxl
except ImportError:   # pinned: newer mlxtend releases force-upgrade pandas/scik
    %pip install -q "mlxtend==0.23.4" openpyxl
import json, platform, re, time, urllib.request, warnings, zipfile
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn, mlxtend

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 40); pd.set_option("display.max_colwidth",
plt.rcParams["figure.figsize"] = (8, 4.5)
print(f"python {platform.python_version()} | pandas {pd.__version__} | scikit-le
```

```
python 3.13.18 | pandas 3.0.5 | scikit-learn 1.9.1 | mlxtend 0.25.0
```

In [2]:
```
# ============================ CONFIGURATION ============================
DATA_URL = "https://archive.ics.uci.edu/static/public/502/online+retail+ii.zip"
DISCOVERY_END = pd.Timestamp("2010-12-01")   # mine patterns on Dec 2009 - Nov 2
                                             # ... and check them on Dec 2010 -
K_RANGE        = range(3, 7)    # candidate numbers of customer segments
MIN_SUPPORT    = 0.01           # itemset must appear in >= 1 % of invoices
MIN_CONFIDENCE = 0.40           # P(consequent | antecedent)
MIN_LIFT       = 3.0            # >= 3x more likely than by chance
RANDOM_STATE   = 42
# ======================================================================

ART = Path("artifacts"); DATA, FIG, REP, APP = (ART / p for p in ("data", "figur
for p in (DATA, FIG, REP, APP):
    p.mkdir(parents=True, exist_ok=True)

def savefig(name):
    plt.tight_layout(); plt.savefig(FIG / f"{name}.png", dpi=120, bbox_inches="t
```

Stage 0 — Understanding the application domain and the KDD goals

Fayyad step 1: learn the domain and the relevant prior knowledge, and identify the end user's goals.

Domain and prior knowledge

The data come from a UK-based, store-less online retailer selling mainly unique all-occasion giftware (tableware, Christmas decorations, bags, children's items). Many customers are **wholesalers** who resell the products, so order sizes vary enormously, and

sales are strongly **seasonal**, peaking in September–November ahead of Christmas. Chen, Sain & Guo (2012) first analysed the 2010-11 part of this data with RFM-based segmentation, which is the established retail practice:

- **RFM** (recency, frequency, monetary value) is the standard, explainable way to group customers by purchasing behaviour; recent, frequent, high-spending customers are the most valuable and most likely to buy again.
- **Market-basket analysis** (Agrawal et al., 1993) finds products bought together; it underpins "customers also bought" recommendations and store/catalogue layout.

End-user goals → discovery goals

End user	Business goal	KDD goal (descriptive, not predictive)
Marketing manager	target campaigns at the right customers	**G1** — discover natural **customer segments** from purchasing behaviour
Merchandising / web team	increase basket size with cross-selling	**G2** — discover **product associations** ("customers who buy X also buy Y")

How the patterns will be evaluated (Fayyad's four criteria)

Criterion	Test used here
Valid	patterns found in the discovery year must still hold in the **following year** (segments predict future spend; rules keep their lift and confidence)
Novel	rules are split into obvious *variants* (same product, other colour/design) and genuinely **cross-product** associations
Potentially useful	each segment gets a concrete action; validated rules feed a recommendation tool
Understandable	segments described in plain words against customer tertiles; rules shown as readable product names

---

Stage 1 — Selection

Fayyad step 2: select the data set, or subset of variables and records, on which discovery will be performed.

In [3]:
```
cache = DATA / "online_retail_II.pkl"
if cache.exists():
    raw = pd.read_pickle(cache)
else:
    t0 = time.time()
    archive = DATA / "online_retail_ii.zip"
    if not archive.exists():
        urllib.request.urlretrieve(DATA_URL, archive)
    with zipfile.ZipFile(archive) as z:
        xlsx = [n for n in z.namelist() if n.endswith(".xlsx")][0]
```

```
        z.extract(xlsx, DATA)
    sheets = pd.read_excel(DATA / xlsx, sheet_name=None)          # BOTH sheets
    print({name: len(s) for name, s in sheets.items()})
    raw = pd.concat(sheets.values(), ignore_index=True)
    raw = raw.rename(columns={"InvoiceNo": "Invoice", "UnitPrice": "Price", "Cus
    raw["Invoice"] = raw["Invoice"].astype(str)
    raw["StockCode"] = raw["StockCode"].astype(str).str.strip().str.upper()
    raw["Description"] = raw["Description"].astype(str).str.strip()
    raw["Customer ID"] = raw["Customer ID"].astype("Int64").astype(str).replace(
    raw["InvoiceDate"] = pd.to_datetime(raw["InvoiceDate"])
    raw.to_pickle(cache)
    print(f"Read Excel in {time.time() - t0:.0f}s (cached for next time)")
print(f"{len(raw):,} transaction lines x {raw.shape[1]} attributes | {raw['Invoi
raw.head()
```

```
{'Year 2009-2010': 502938, 'Year 2010-2011': 564433}
Read Excel in 19s (cached for next time)
1,067,371 transaction lines x 8 attributes | 2009-12-01 07:45:00 -> 2011-12-09 1
2:50:00
```

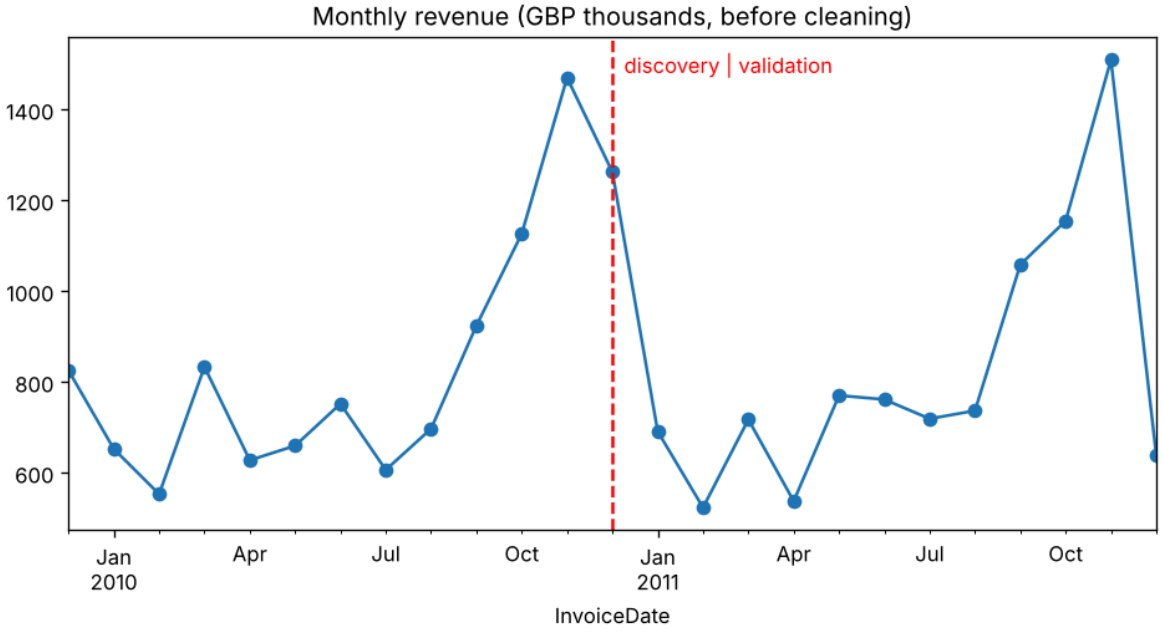
Out[3]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085	United Kingdom

In [4]:

```
overview = pd.DataFrame({
    "lines": [len(raw)], "invoices": [raw["Invoice"].nunique()], "customers": [r
    "products": [raw["StockCode"].nunique()], "countries": [raw["Country"].nuniq
    "cancellation_lines": [raw["Invoice"].str.startswith("C").sum()],
    "lines_without_customer_%": [round(raw["Customer ID"].isna().mean() * 100, 1
display(overview)
monthly = raw[~raw["Invoice"].str.startswith("C")].assign(revenue=lambda d: d["Q
            .set_index("InvoiceDate").resample("MS")["revenue"].sum() / 1e3
ax = monthly.plot(marker="o", title="Monthly revenue (GBP thousands, before clea
ax.axvline(DISCOVERY_END, c="r", ls="--"); ax.text(DISCOVERY_END, ax.get_ylim()[
savefig("selection_monthly_revenue")
```

	lines	invoices	customers	products	countries	cancellation_lines	lines_without_cus
0	1067371	53628	5942	5131	43	19494	



Target data set

Decision	Choice	Rationale
Records	all transaction lines, Dec 2009 – Dec 2011	full history available
Attributes	all 8 (invoice, product code, description, quantity, date, price, customer, country)	all are needed for RFM and baskets
Discovery period	1 Dec 2009 – 30 Nov 2010 (12 months)	one full seasonal cycle to mine patterns
Validation period	1 Dec 2010 – 9 Dec 2011	untouched year to test whether patterns are **valid**
Segmentation scope	lines with a Customer ID	RFM needs a customer
Basket scope	all invoices (with or without Customer ID)	an anonymous basket is still a basket

🔍 Critic review — Selection

Paste this stage (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Selection** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether the target data set matches the KDD goals, whether the discovery/validation split is justified, and whether the scoping decisions (e.g. anonymous transactions) are explained. List (1)*

> *anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	23 % of lines have no Customer ID and are excluded from segmentation; discovery/validation periods must each cover a season.	Stated the limitation (those lines are still used for baskets); confirmed both periods span a full Dec–Nov cycle.
2	`<fresh-chat critique>`	`<changes>`
3		

Stage 2 — Preprocessing

Fayyad step 3: basic operations such as removing noise, deciding how to handle missing data, and accounting for known changes.

In [5]:
```
df = raw.copy(); log = []
def step(action, mask_removed):
    global df
    log.append({"action": action, "lines_removed": int(mask_removed.sum()), "lin
    df = df[~mask_removed]

is_cancel = df["Invoice"].str.startswith("C")
cancellations = df[is_cancel].copy()                        # kept aside for a re
step("Separate cancellation lines (invoice starts with 'C')", is_cancel)

# Remove purchases that were later cancelled in full (same customer, product and
cancel_keys = set(zip(cancellations["Customer ID"], cancellations["StockCode"],
cancelled_purchase = pd.Series([k in cancel_keys for k in zip(df["Customer ID"],
step("Remove purchases matched by a later cancellation", cancelled_purchase)

non_product = ~df["StockCode"].str.match(r"^\d{5}")         # POST, DOT, M, BANK
print("Non-product codes removed:", sorted(df.loc[non_product, "StockCode"].uniq
step("Remove non-product codes (postage, fees, manual adjustments, tests)", non_
step("Remove lines with quantity <= 0 or price <= 0", (df["Quantity"] <= 0) | (d
step("Remove exact duplicate lines", df.duplicated())

# One description per product (codes have spelling variants / damaged-stock note
canonical = df.groupby("StockCode")["Description"].agg(lambda s: s.value_counts(
df["Description"] = df["StockCode"].map(canonical)
df["Revenue"] = df["Quantity"] * df["Price"]

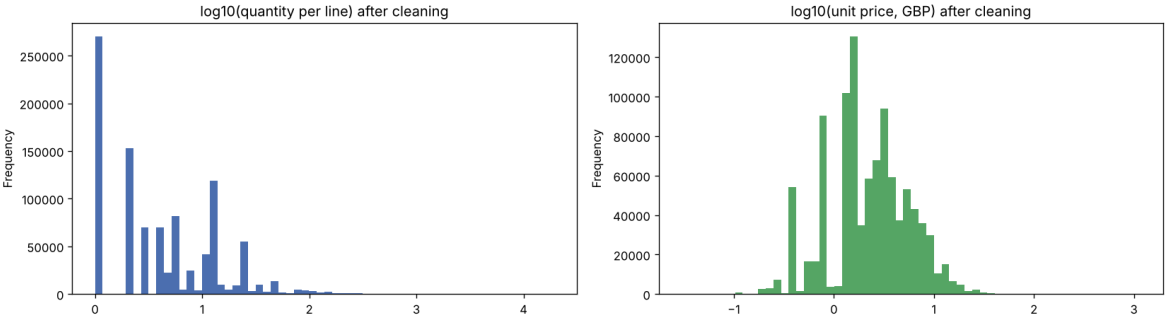
preprocessing_log = pd.DataFrame(log); preprocessing_log.to_csv(REP / "preproces
display(preprocessing_log)
print(f"Clean: {len(df):,} lines, {df['Invoice'].nunique():,} invoices, {df['Sto
      f"{df['Customer ID'].nunique():,} customers; {df['Customer ID'].isna().mea
```

```
Non-product codes removed: ['ADJUST', 'ADJUST2', 'AMAZONFEE', 'B', 'BANK CHARGE
S', 'C2', 'C3', 'D', 'DCGS0003', 'DCGS0004', 'DCGS0006', 'DCGS0016', 'DCGS0027',
'DCGS0036', 'DCGS0037', 'DCGS0039', 'DCGS0041', 'DCGS0044', 'DCGS0053', 'DCGS005
5', 'DCGS0056', 'DCGS0057', 'DCGS0058', 'DCGS0059', 'DCGS0060'] ...
```

	action	lines_removed	lines_left
0	Separate cancellation lines (invoice starts with 'C')	19494	1047877
1	Remove purchases matched by a later cancellation	13653	1034224
2	Remove non-product codes (postage, fees, manual adjustme...	4320	1029904
3	Remove lines with quantity <= 0 or price <= 0	6111	1023793
4	Remove exact duplicate lines	33203	990590

```
Clean: 990,590 lines, 39,009 invoices, 4,699 products, 5,830 customers; 22.9% of
lines have no customer
```

In [6]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
np.log10(df["Quantity"]).plot.hist(bins=60, ax=axes[0], color="#4C72B0", title="
np.log10(df["Price"]).plot.hist(bins=60, ax=axes[1], color="#55A868", title="log
savefig("preprocessing_distributions")
display(df.nlargest(5, "Revenue")[["Invoice", "InvoiceDate", "Customer ID", "Des
```



	Invoice	InvoiceDate	Customer ID	Description	Quantity	Price	Revenue
228042	511465	2010-06-08 12:59:00	18008	PINK PAPER PARASOL	3500	2.55	8925.00
873786	567423	2011-09-20 11:05:00	17450	SET OF TEA COFFEE SUGAR TINS PANTRY	1412	5.06	7144.72
461644	533027	2010-11-15 16:02:00	NaN	PAPER CHAIN KIT 50'S CHRISTMAS	835	6.95	5803.25
379875	525968	2010-10-08 10:10:00	15838	ROTATING SILVER ANGELS T-LIGHT HLDR	3120	1.66	5179.20
947062	573003	2011-10-27 12:11:00	14646	RABBIT NIGHT LIGHT	2400	2.08	4992.00

Preprocessing decisions

- **Cancellations (19,494 lines) are separated, not netted.** A cancellation is a different event from a purchase; netting would hide it. Purchases that were later cancelled in

full (same customer, product and quantity — 13,653 lines) are removed so they don't count as demand. Partial returns cannot be matched this way; they are captured instead by each customer's `return_rate`.

- **Non-product codes (4,320 lines)** — postage (`POST`, `DOT`), bank charges, Amazon fees, manual adjustments (`M`, `ADJUST`), samples, tests and `DCGS...` gift codes — are not products and would create meaningless rules ("postage → postage").
- **Zero/negative quantity or price (6,111 lines)** are stock corrections, damaged/lost goods and free items, not sales.
- **Exact duplicates (33,203 lines)** — identical invoice, product, quantity, price and timestamp — are treated as double entries. Baskets are binary, so rules are unaffected; monetary values shrink slightly.
- **Missing Customer ID (22.9 % of clean lines):** kept for basket analysis (a basket is a basket), excluded from segmentation (RFM needs a customer). This is a limitation: if anonymous buyers behave differently, the segments describe registered customers only.
- **Extreme lines:** the largest lines (e.g. 3,500 paper parasols for £8,925) look like genuine wholesale orders, not errors; they are kept because the log-transformation in Stage 3 limits their influence on clustering.

Result: 990,590 clean lines, 39,009 invoices, 4,699 products, 5,830 identified customers.

🔍 Critic review — Preprocessing

Paste this stage (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Preprocessing** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether every cleaning rule is justified by an observed data problem, how missing customer IDs and cancellations are handled, and whether any rule could silently bias the segments or rules. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Cancellations matched only on exact customer/product/quantity; are the 33k duplicate lines really errors?	Partial returns captured by `return_rate`; duplicates identical to the second, baskets binary so rules unaffected — documented.
2	`<fresh-chat critique>`	`<changes>`
3		

Stage 3 — Transformation

Fayyad step 4: find useful features to represent the data, using dimensionality reduction or transformation methods.

3.1 Customer table (for G1): RFM and behavioural features

In [7]:
```
def customer_features(lines, cancels, snapshot):
    """One row per customer, measured at `snapshot` (the day after the period en
    c = lines.dropna(subset=["Customer ID"]).groupby("Customer ID")
    f = pd.DataFrame({
        "recency_days": (snapshot - c["InvoiceDate"].max()).dt.days,
        "frequency": c["Invoice"].nunique(),
        "monetary": c["Revenue"].sum(),
        "tenure_days": (snapshot - c["InvoiceDate"].min()).dt.days,
        "distinct_products": c["StockCode"].nunique(),
        "country": c["Country"].agg(lambda s: s.mode()[0]),
    })
```

```
    f["avg_basket_value"] = f["monetary"] / f["frequency"]
    returned = (cancels.assign(v=-cancels["Quantity"] * cancels["Price"]).groupb
    f["return_rate"] = (returned.reindex(f.index).fillna(0) / f["monetary"]).cli
    return f

disc_lines, val_lines = df[df["InvoiceDate"] < DISCOVERY_END], df[df["InvoiceDat
disc_cancels = cancellations[cancellations["InvoiceDate"] < DISCOVERY_END]
customers = customer_features(disc_lines, disc_cancels, DISCOVERY_END)
print(f"{len(customers):,} customers in the discovery period"); display(customer
```

```
4,218 customers in the discovery period
```

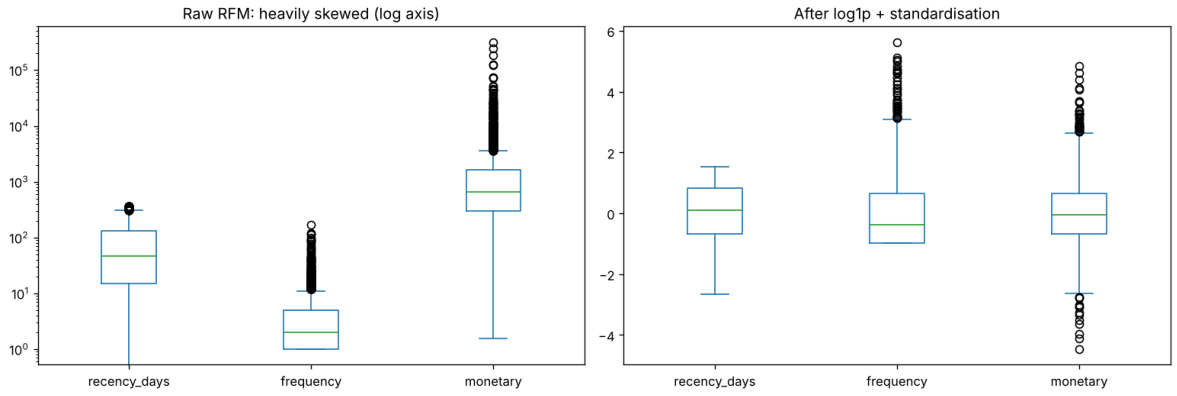
	count	mean	std	min	25%	50%	75%	max
recency_days	4218.0	86.6	94.6	0.0	15.0	47.0	132.0	364.0
frequency	4218.0	4.3	7.4	1.0	1.0	2.0	5.0	172.0
monetary	4218.0	1903.1	8211.9	1.6	303.6	667.0	1629.4	319471.3
tenure_days	4218.0	217.8	117.3	0.0	113.0	244.5	320.0	364.0
distinct_products	4218.0	62.6	83.6	1.0	17.0	38.0	77.8	1690.0
avg_basket_value	4218.0	373.7	502.8	1.6	177.8	285.6	420.7	11880.8
return_rate	4218.0	0.0	0.1	0.0	0.0	0.0	0.0	1.0

In [8]:

```
from sklearn.preprocessing import FunctionTransformer, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA

SEGMENT_FEATURES = ["recency_days", "frequency", "monetary"]          # classic
projector = Pipeline([("log", FunctionTransformer(np.log1p)), ("scale", Standard
Z = projector.fit_transform(customers[SEGMENT_FEATURES])

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
customers[SEGMENT_FEATURES].plot.box(ax=axes[0], logy=True, title="Raw RFM: heav
pd.DataFrame(Z, columns=SEGMENT_FEATURES).plot.box(ax=axes[1], title="After log1
savefig("transformation_rfm")
print("Correlation of transformed RFM features:"); display(pd.DataFrame(Z, colum
pca = PCA(2).fit(Z); print(f"2-D PCA projection keeps {pca.explained_variance_ra
```




```
Correlation of transformed RFM features:
```

	recency_days	frequency	monetary
recency_days	1.00	-0.48	-0.43
frequency	-0.48	1.00	0.81
monetary	-0.43	0.81	1.00

```
2-D PCA projection keeps 94% of the variance (used for plots)
```

Why these transformations. Frequency and monetary value span several orders of magnitude (1 to 172 orders; £2 to £319k per customer) and are right-skewed, while K-means uses Euclidean distance, so a handful of wholesalers would dominate untransformed data. `log1p` compresses the scale and standardisation gives each RFM dimension equal weight. Only three features are used so each segment can be explained in one sentence. Caveat: frequency and monetary still correlate at 0.81 after transformation, so purchase *volume* is effectively weighted twice — an iteration could drop one of them or cluster on principal components. The 2-D PCA projection keeps 94 % of the variance, so the plots in Stage 5 are faithful.

3.2 Basket matrix (for G2): invoice × product

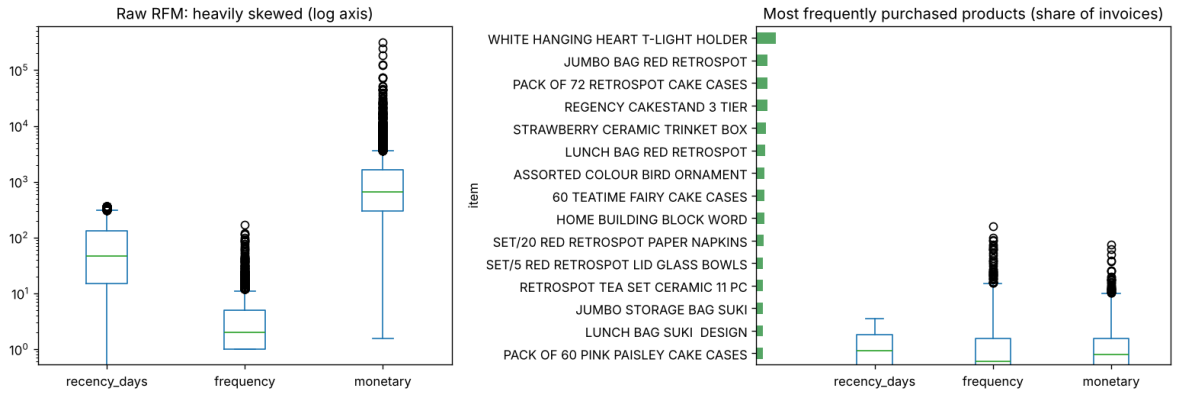
In [9]:
```
item_name = df.drop_duplicates("StockCode").set_index("StockCode")["Description"
dup = item_name.duplicated(keep=False)
item_name[dup] = item_name[dup] + " (" + item_name[dup].index + ")"        # kee

def item_support(lines):
    """Share of invoices that contain each product."""
    l = lines.assign(item=lines["StockCode"].map(item_name))
    return (l.groupby("item")["Invoice"].nunique() / l["Invoice"].nunique()).sor

def basket_matrix(lines, items):
    """Boolean invoice x product matrix restricted to `items`; keeps EVERY invoi
    l = lines.assign(item=lines["StockCode"].map(item_name))
    b = l[l["item"].isin(items)].groupby(["Invoice", "item"]).size().unstack(fil
    return b.reindex(index=l["Invoice"].unique(), columns=items, fill_value=Fals

disc_support = item_support(disc_lines)
frequent_items = disc_support[disc_support >= MIN_SUPPORT].index
disc_baskets = basket_matrix(disc_lines, frequent_items)
print(f"{disc_lines['Invoice'].nunique():,} invoices x {disc_lines['StockCode'].
      f"with support >= {MIN_SUPPORT:.0%}; matrix {disc_baskets.shape[0]:,} x {d
disc_support.head(15).sort_values().plot.barh(color="#55A868", title="Most frequ
savefig("transformation_item_support")
```

```
19,482 invoices x 4,047 products -> reduced to 717 products with support >= 1%; m
atrix 19,482 x 717 (14 MB)
```



Why this reduction is safe. An itemset can only be frequent if every item in it is frequent (the *Apriori property*). Removing the 3,330 products that appear in fewer than 1 % of invoices therefore loses no possible rule, and shrinks the matrix from 4,047 to 717 products (14 MB), which makes FP-Growth run in seconds. Every invoice is kept, so support is measured against all 19,482 discovery-year baskets.

🔍 Critic review — Transformation

Paste this stage (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Transformation** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether the chosen representations suit each mining goal, whether scaling and reduction choices are justified, and whether information that matters was lost. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Frequency and monetary correlate 0.81 after transformation, double-weighting volume.	Kept RFM for explainability; recorded the caveat and a PCA/drop-one alternative as a next iteration.
2	`<fresh-chat critique>`	`<changes>`
3		

Stage 4 — Data mining

Fayyad steps 5–7: match the goals to a mining method, choose the algorithms and parameters, and search for patterns.

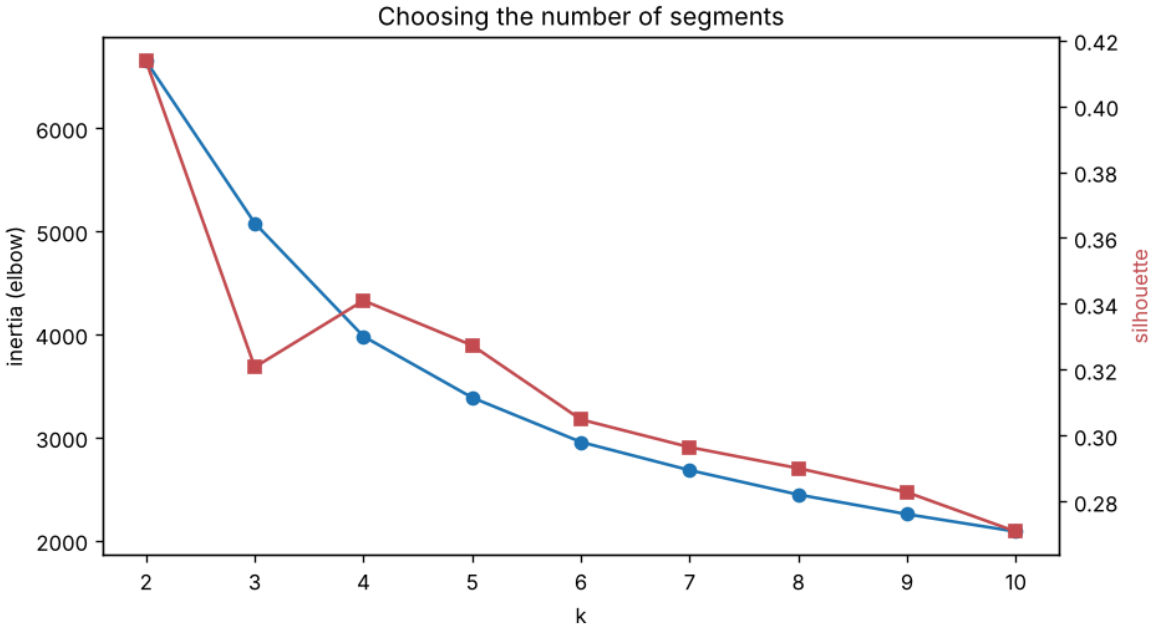
Goal	Method (step 5)	Algorithm (step 6)	Why
G1 segments	clustering	**K-means** on log-RFM	fast, simple to explain, centroids are readable profiles; *k* chosen with silhouette + elbow
G2 associations	association-rule mining	**FP-Growth** + rule generation	same output as Apriori but without candidate generation — much faster on wide baskets

4.1 Customer segmentation (step 7, G1)

In [10]:
```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

k_search = []
for k in range(2, 11):
    km = KMeans(k, n_init=10, random_state=RANDOM_STATE).fit(Z)
    k_search.append({"k": k, "inertia": km.inertia_, "silhouette": silhouette_sc
k_search = pd.DataFrame(k_search).round(4); k_search.to_csv(REP / "k_selection.c
fig, ax = plt.subplots(); ax.plot(k_search["k"], k_search["inertia"], marker="o"
ax2 = ax.twinx(); ax2.plot(k_search["k"], k_search["silhouette"], marker="s", c=
ax.set_title("Choosing the number of segments"); savefig("mining_k_selection")

K = int(k_search[k_search["k"].isin(K_RANGE)].sort_values("silhouette", ascendin
print(f"Chosen k = {K} (best silhouette within {list(K_RANGE)} - fewer than 3 se
```




```
Chosen k = 4 (best silhouette within [3, 4, 5, 6] - fewer than 3 segments is too
coarse to act on)
```

In [11]:
```
segmenter = Pipeline([("project", projector), ("kmeans", KMeans(K, n_init=20, ra
customers["cluster"] = segmenter.predict(customers[SEGMENT_FEATURES])

# Stability: do bootstrap resamples find the same segments? (adjusted Rand index
```

```
from sklearn.metrics import adjusted_rand_score
aris = []
for b in range(10):
    boot = customers.sample(frac=1, replace=True, random_state=b)
    m = Pipeline([("project", projector), ("kmeans", KMeans(K, n_init=10, random
    aris.append(adjusted_rand_score(customers["cluster"], m.predict(customers[SE
print(f"Bootstrap stability (ARI over 10 resamples): mean {np.mean(aris):.3f}, m
```

```
Bootstrap stability (ARI over 10 resamples): mean 0.938, min 0.902
```

4.2 Association rules (step 7, G2)

In [12]:

```
from mlxtend.frequent_patterns import fpgrowth, association_rules
t0 = time.time()
itemsets = fpgrowth(disc_baskets, min_support=MIN_SUPPORT, use_colnames=True, ma
try:
    rules = association_rules(itemsets, num_itemsets=len(disc_baskets), metric="
except TypeError:                                   # older mlxtend versions hav
    rules = association_rules(itemsets, metric="confidence", min_threshold=MIN_C
rules = rules[rules["lift"] >= MIN_LIFT].copy()
rules["antecedents"] = rules["antecedents"].apply(lambda s: tuple(sorted(s)))
rules["consequents"] = rules["consequents"].apply(lambda s: tuple(sorted(s)))
rules = rules.sort_values(["lift", "confidence"], ascending=False).reset_index(d
print(f"{len(itemsets):,} frequent itemsets -> {len(rules):,} rules with confide
display(rules[["antecedents", "consequents", "support", "confidence", "lift"]].h
```

```
1,389 frequent itemsets -> 579 rules with confidence >= 0.4 and lift >= 3.0 (3s)
```

	antecedents	consequents	support	confidence	lift
0	(CHILDS GARDEN TROWEL PINK,)	(CHILDS GARDEN FORK PINK,)	0.010	0.755	68.071
1	(CHILDS GARDEN FORK PINK,)	(CHILDS GARDEN TROWEL PINK,)	0.010	0.926	68.071
2	(CHILDS GARDEN TROWEL PINK,)	(CHILDS GARDEN TROWEL BLUE,)	0.011	0.796	60.832
3	(CHILDS GARDEN TROWEL BLUE,)	(CHILDS GARDEN TROWEL PINK,)	0.011	0.827	60.832
4	(POPPY'S PLAYHOUSE LIVINGROOM,)	(POPPY'S PLAYHOUSE BEDROOM,)	0.010	0.852	59.260
5	(POPPY'S PLAYHOUSE BEDROOM,)	(POPPY'S PLAYHOUSE LIVINGROOM,)	0.010	0.718	59.260
6	(POPPY'S PLAYHOUSE LIVINGROOM,)	(POPPY'S PLAYHOUSE KITCHEN,)	0.011	0.898	57.758
7	(POPPY'S PLAYHOUSE KITCHEN,)	(POPPY'S PLAYHOUSE LIVINGROOM,)	0.011	0.700	57.758
8	(POPPY'S PLAYHOUSE BEDROOM,)	(POPPY'S PLAYHOUSE KITCHEN,)	0.012	0.861	55.341
9	(POPPY'S PLAYHOUSE KITCHEN,)	(POPPY'S PLAYHOUSE BEDROOM,)	0.012	0.795	55.341

🔍 Critic review — Data mining

Paste this stage (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Data mining** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether each algorithm matches its goal, whether the parameter choices (k, minimum support, confidence, lift) are justified rather than arbitrary, and whether the stability of the segments was demonstrated. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Silhouette is best at k = 2; restricting to 3–6 needs a reason.	Justified on usefulness; k = 4 best within range (0.341); bootstrap ARI 0.94 shows stability.
2	`<fresh-chat critique>`	`<changes>`
3		

Stage 5 — Interpretation / Evaluation

Fayyad step 8: interpret the mined patterns, possibly returning to earlier steps, and judge them against the criteria set in Stage 0.

5.1 Segment profiles (understandable)

In [13]:
```
# Critique fix: describe segments against customer TERTILES (a median split call
q = customers[SEGMENT_FEATURES + ["tenure_days"]].quantile([1 / 3, 2 / 3])
def level(v, col, low, mid, high):
    return low if v <= q.loc[1 / 3, col] else (high if v > q.loc[2 / 3, col] els
def describe_segment(r):
    parts = [level(r["recency_days"], "recency_days", "active", "cooling", "laps
             level(r["frequency"], "frequency", "one-off", "repeat", "frequent")
             level(r["monetary"], "monetary", "low-spend", "mid-spend", "high-sp
    return " · ".join(parts + (["new"] if r["tenure_days"] <= 90 else []))

prof = customers.groupby("cluster").agg(customers=("frequency", "size"), recency
                                        frequency=("frequency", "median"), monet
                                        avg_basket_value=("avg_basket_value", "m
                                        return_rate=("return_rate", "mean"), rev
prof["revenue_share_%"] = (prof["revenue"] / prof["revenue"].sum() * 100).round(
prof["customer_share_%"] = (prof["customers"] / prof["customers"].sum() * 100).r
prof = prof.sort_values("monetary", ascending=False)
```

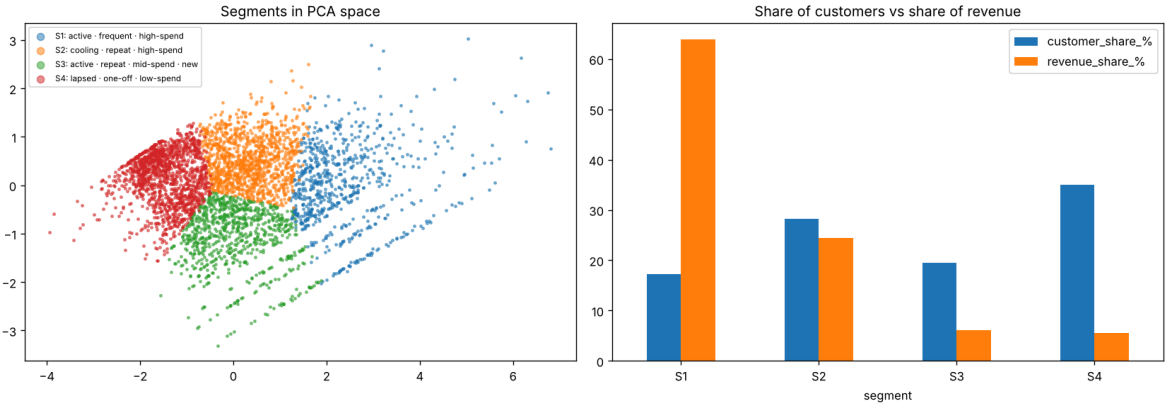
```
prof["profile"] = prof.apply(describe_segment, axis=1)
prof["segment"] = [f"S{i + 1}" for i in range(len(prof))]
SEGMENT_OF = prof["segment"].to_dict(); customers["segment"] = customers["cluste
prof = prof.set_index("segment").round(2); prof.to_csv(REP / "segment_profiles.c
print("Tertile cut-offs used for the labels:"); display(q.round(1))
display(prof.drop(columns="revenue"))
```

```
Tertile cut-offs used for the labels:
```

	recency_days	frequency	monetary	tenure_days
0.333333	23.0	1.0	385.4	169.0
0.666667	86.0	4.0	1220.9	295.0

segment	customers	recency_days	frequency	monetary	avg_basket_value	tenure_days
S1	726	8.0	9.0	3447.20	353.08	350.0
S2	1193	57.0	4.0	1293.03	347.98	280.0
S3	821	12.0	2.0	492.36	250.35	57.0
S4	1478	166.0	1.0	259.38	206.48	200.5

In [14]:
```
xy = PCA(2).fit(Z).transform(Z)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for s, g in customers.assign(pc1=xy[:, 0], pc2=xy[:, 1]).groupby("segment"):
    axes[0].scatter(g["pc1"], g["pc2"], s=4, alpha=.5, label=f"{s}: {prof.loc[s,
axes[0].legend(fontsize=7, markerscale=3); axes[0].set_title("Segments in PCA sp
prof[["customer_share_%", "revenue_share_%"]].plot.bar(ax=axes[1], rot=0, title=
savefig("evaluation_segments")
```



Reading the profiles (medians, discovery year; labels are relative to customer tertiles — recency 23/86 days, frequency 1/4 orders, spend £385/£1,221):

Segment	Who they are	Share of customers → revenue
S1 · Champions	ordered 8 days ago, 9 orders, £3,447 spent, customers for ~a year	17 % → **64 %**
S2 · Loyal but cooling	4 orders and £1,293 spent, but last order 57 days ago	28 % → 25 %
S3 · New customers	active (12 days) but only ~2 months' tenure, 2 orders, £492	20 % → 6 %
S4 · Lapsed one-off buyers	one order, £259, 166 days ago	35 % → 6 %

Return rates are similar (2–3 %) across segments, so returns do not distinguish them. The actions below are written for these four groups.

In [15]:
```
# Segment names and actions, written after reading the profiles and the next-yea
SEGMENT_ACTIONS = {
    "S1": {"name": "Champions",
           "action": "Protect: account manager or VIP tier, early access to new
    "S2": {"name": "Loyal but cooling",
           "action": "Re-engage: personalised reminder with best-selling reorder
    "S3": {"name": "New customers",
           "action": "Develop: onboarding emails, cross-sell from the associatio
    "S4": {"name": "Lapsed one-off buyers",
           "action": "Low-cost win-back only (seasonal newsletter before Q4); do
}
for s in SEGMENT_ACTIONS:
    SEGMENT_ACTIONS[s]["name"] = f"{s} · {SEGMENT_ACTIONS[s]['name']} ({prof.loc
pd.DataFrame(SEGMENT_ACTIONS).T
```

Out[15]:

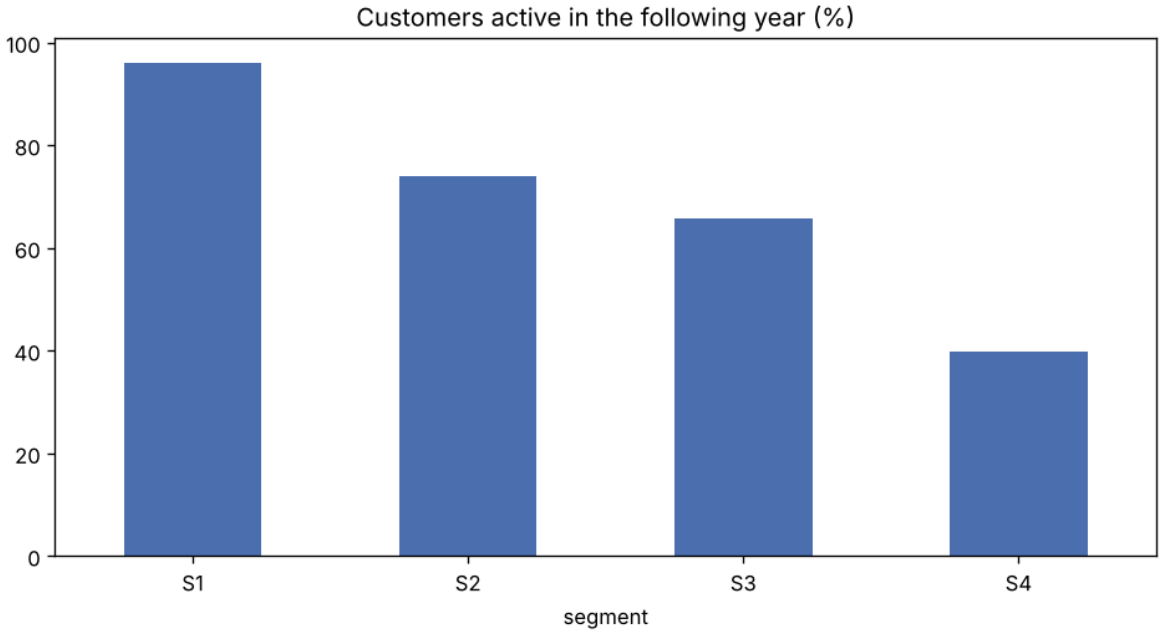
	name	action
S1	S1 · Champions (active · frequent · high-spend)	Protect: account manager or VIP tier, early access to ne...
S2	S2 · Loyal but cooling (cooling · repeat · high-spend)	Re-engage: personalised reminder with best-selling reord...
S3	S3 · New customers (active · repeat · mid-spend · new)	Develop: onboarding emails, cross-sell from the associat...
S4	S4 · Lapsed one-off buyers (lapsed · one-off · low-spend)	Low-cost win-back only (seasonal newsletter before Q4); ...

5.2 Are the segments valid? Behaviour in the following year

In [16]:
```
nxt = val_lines.dropna(subset=["Customer ID"]).groupby("Customer ID")["Revenue"]
validity = customers.assign(next_year_revenue=nxt.reindex(customers.index).filln
    customers=("next_year_revenue", "size"),
```

```
    active_next_year_pct=("next_year_revenue", lambda s: round((s > 0).mean() *
    median_next_year_revenue=("next_year_revenue", "median"),
    total_next_year_revenue=("next_year_revenue", "sum")).reindex(prof.index)
validity["share_of_next_year_revenue_%"] = (validity["total_next_year_revenue"]
validity.to_csv(REP / "segment_validity_next_year.csv"); display(validity.round(
validity[["active_next_year_pct"]].plot.bar(rot=0, legend=False, color="#4C72B0"
savefig("evaluation_segment_validity")
```

segment	customers	active_next_year_pct	median_next_year_revenue	total_next_year_reve
S1	726	96.0	2557.7	448870
S2	1193	73.9	574.1	138399
S3	821	65.8	256.4	48505
S4	1478	39.9	0.0	36253



The segments are valid. Segments were built only from 2009-10 behaviour, yet they sharply separate who buys in 2010-11: 96 % of Champions bought again (median £2,558 each, 67 % of next year's revenue), against 74 % of S2, 66 % of New customers and 40 % of Lapsed one-off buyers (median £0). The ordering of the segments is exactly the ordering of their future value, so they capture something real about customers rather than noise. It also shows where marketing money matters: S2 still produced 21 % of next year's revenue, so reactivating "cooling" customers is worth more than chasing lapsed ones.

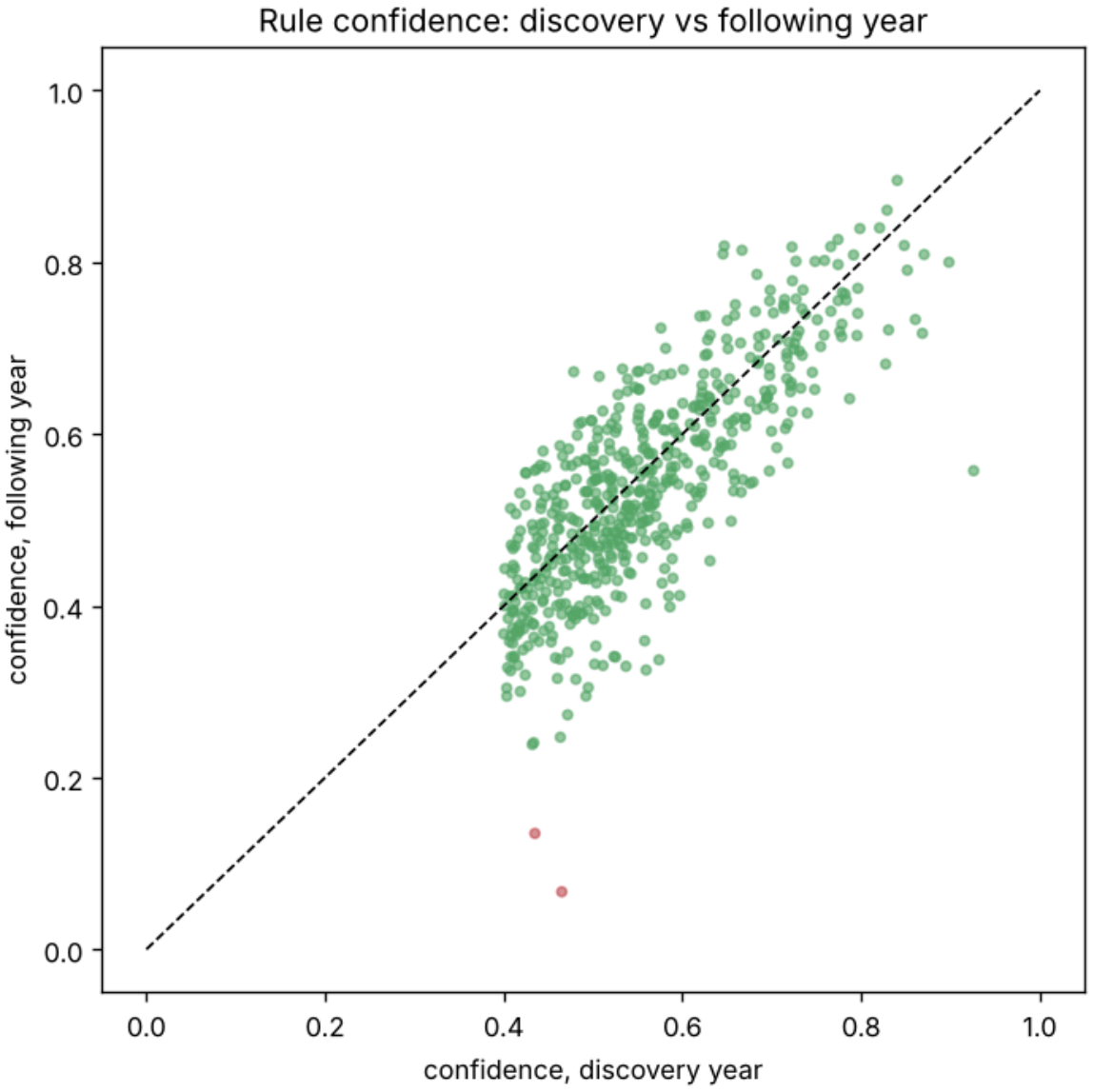
5.3 Are the rules valid? Re-measure each rule on the following year

In [17]:
```
val_baskets = basket_matrix(val_lines, items=frequent_items)
def rule_stats(baskets, a, c):
    has_a = baskets[list(a)].all(axis=1); has_c = baskets[list(c)].all(axis=1)
    sup_a, sup_c, sup_ac = has_a.mean(), has_c.mean(), (has_a & has_c).mean()
```

```
    conf = sup_ac / sup_a if sup_a else np.nan
    return pd.Series({"val_support": sup_ac, "val_confidence": conf, "val_lift":

rules = rules.join(rules.apply(lambda r: rule_stats(val_baskets, r["antecedents"
rules["holds_next_year"] = (rules["val_lift"] >= MIN_LIFT / 2) & (rules["val_con
print(f"{rules['holds_next_year'].mean():.0%} of rules still hold in the followi
      f"(lift >= {MIN_LIFT / 2}, confidence >= {MIN_CONFIDENCE / 2})")
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(rules["confidence"], rules["val_confidence"], c=rules["holds_next_yea
ax.plot([0, 1], [0, 1], "k--", lw=1); ax.set(xlabel="confidence, discovery year"
                                             title="Rule confidence: discovery
savefig("evaluation_rule_validity")
```

```
99% of rules still hold in the following year (lift >= 1.5, confidence >= 0.2)
```



In [18]:
```
# Critique fix: the halved thresholds above are lenient - also apply the ORIGINA
rules["holds_strict"] = (rules["val_lift"] >= MIN_LIFT) & (rules["val_confidence
lift_ratio = (rules["val_lift"] / rules["lift"]).describe(percentiles=[.1, .5, .
print(f"Strict test (lift >= {MIN_LIFT} and confidence >= {MIN_CONFIDENCE} again
      f"{rules['holds_strict'].mean():.0%} of rules hold")
print("Next-year lift / discovery lift:"); display(lift_ratio[["10%", "50%", "90
```

```
Strict test (lift >= 3.0 and confidence >= 0.4 again next year): 86% of rules hol
d
Next-year lift / discovery lift:
```

	ratio
10%	0.83
50%	1.05
90%	1.37

5.4 Are the rules novel? Obvious variants vs cross-category associations

In [19]:
```
# Critique fix (novelty): a rule is an obvious VARIANT if all its products are t
# colour/design/size (e.g. KEY FOB GARAGE vs KEY FOB SHED); otherwise it links d
STYLE = r"\b(RED|PINK|BLUE|GREEN|WHITE|BLACK|IVORY|CREAM|PURPLE|YELLOW|ORANGE|SI
words = lambda name: re.findall(r"[A-Z]{2,}", re.sub(STYLE, " ", name.upper()))
def same_product(a, b):
    wa, wb = words(a), words(b)
    if len(wa) >= 2 and len(wb) >= 2 and (wa[:2] == wb[:2] or wa[-2:] == wb[-2:]
        return True
    sa, sb = set(wa), set(wb)
    return len(sa & sb) / max(len(sa | sb), 1) >= 0.5

from itertools import combinations
rules["rule_type"] = rules.apply(lambda r: "variant (same product, other design)
                                 if all(same_product(a, b) for a, b in combinati
                                 else "cross-product", axis=1)
display(rules.groupby("rule_type").agg(rules=("lift", "size"), median_lift=("lif
                                       holds_strict_pct=("holds_strict", lambda
show = ["antecedents", "consequents", "support", "confidence", "lift", "val_conf
print("Strongest cross-product rules that met the full thresholds again in the f
display(rules[(rules["rule_type"] == "cross-product") & rules["holds_strict"]][s
rules.to_csv(REP / "association_rules_evaluated.csv", index=False)
```

rule_type	rules	median_lift	holds_strict_pct
cross-product	154	9.541569	74.7
variant (same product, other design)	425	12.032790	90.1

```
Strongest cross-product rules that met the full thresholds again in the following
year:
```

	antecedents	consequents	support	confidence	lift	val_confidence	val
28	(SET/20 RED RETROSPOT PAPER NAPKINS, SET/6 RED SPOTTY PA...	(SET/6 RED SPOTTY PAPER PLATES,)	0.011	0.840	35.987	0.895	33.
29	(SET/6 RED SPOTTY PAPER PLATES,)	(SET/20 RED RETROSPOT PAPER NAPKINS, SET/6 RED SPOTTY PA...	0.011	0.475	35.987	0.503	33.
40	(SET/20 RED RETROSPOT PAPER NAPKINS, SET/6 RED SPOTTY PA...	(SET/6 RED SPOTTY PAPER CUPS,)	0.011	0.692	32.816	0.716	32.
41	(SET/6 RED SPOTTY PAPER CUPS,)	(SET/20 RED RETROSPOT PAPER NAPKINS, SET/6 RED SPOTTY PA...	0.011	0.526	32.816	0.619	32.
48	(SET/6 RED SPOTTY PAPER CUPS,)	(SET/6 RED SPOTTY PAPER PLATES,)	0.015	0.723	30.941	0.818	30.
49	(SET/6 RED SPOTTY PAPER PLATES,)	(SET/6 RED SPOTTY PAPER CUPS,)	0.015	0.653	30.941	0.664	30.
81	(RED RETROSPOT PLATE,)	(RED RETROSPOT CUP,)	0.012	0.648	25.088	0.587	29.
82	(RED RETROSPOT CUP,)	(RED RETROSPOT PLATE,)	0.012	0.453	25.088	0.458	29.
88	(BLUE POLKADOT BOWL,)	(BLUE POLKADOT CUP,)	0.010	0.547	24.317	0.517	31.
89	(BLUE POLKADOT CUP,)	(BLUE POLKADOT BOWL,)	0.010	0.454	24.317	0.508	31.
107	(PINK POLKADOT BOWL,)	(PINK POLKADOT CUP,)	0.011	0.501	22.394	0.492	31.
108	(PINK POLKADOT CUP,)	(PINK POLKADOT BOWL,)	0.011	0.491	22.394	0.533	31.
124	(RED RETROSPOT PLATE,)	(RED RETROSPOT BOWL,)	0.010	0.554	21.079	0.487	21.
135	(SET OF 20 VINTAGE CHRISTMAS NAPKINS,)	(60 CAKE CASES VINTAGE CHRISTMAS,)	0.013	0.542	20.242	0.437	13.

Evaluation against the Stage 0 criteria

Criterion	Evidence	Verdict
Valid	Segments: next-year activity 96 % / 74 % / 66 % / 40 %, in the predicted order; bootstrap ARI 0.94. Rules: 86 % meet the *full* thresholds (lift ≥ 3, confidence ≥ 0.4) again next year; median lift changes by only +5 %.	✅
Novel	425 of the 579 rules (73 %) are obvious variants (same product, other colour/design) — expected, and still useful for "complete the set" prompts. The 154 cross-product rules (75 % of which hold strictly next year) reveal that customers buy **matching-design sets across product types** (red spotty napkins + cups → plates; polkadot bowls → cups) and **themed bundles** (vintage Christmas napkins → Christmas cake cases). The "new customer" segment (S3) was not anticipated by a plain RFM view.	✅ partly
Potentially useful	Each segment has an action; 396 strictly validated rules drive the recommendation tab of the app.	✅
Understandable	Segments described in plain words against tertiles; rules shown as product names with confidence and lift.	✅

Iterations made (KDD loops back): the first evaluation used median-split labels (mislabelling S3 as "frequent"), lenient validity thresholds (99 % "held") and a word-matching novelty test that called key-fob variants cross-category. All three were corrected after the critic review — see `critiques/round_1.md`.

🔍 Critic review — Interpretation and evaluation

Paste this stage (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Interpretation and evaluation** stage of my retail KDD project below. Check it against everything this KDD stage requires. Focus especially on: whether the patterns were genuinely tested on data not used to find them, whether novelty and usefulness are argued rather than asserted, and whether loops back to earlier stages were considered. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Median-split labels mislabel S3; validity thresholds halved (too lenient); novelty test calls key-fob variants cross-category.	Tertile labels + 'new' flag; strict validity test (86 % hold); product-type novelty test; app uses only strictly validated rules.
2	`<fresh-chat critique>`	`<changes>`
3		

Step 9 — Acting on the discovered knowledge

*Fayyad step 9: use the knowledge directly, incorporate it into another system, or document and report it. Patterns are re-mined on **both years** (now validated) and delivered as a lookup app for marketing and merchandising staff.*

In [20]:
```
import joblib
snapshot = df["InvoiceDate"].max().normalize() + pd.Timedelta(days=1)
final_customers = customer_features(df, cancellations, snapshot)
final_segmenter = Pipeline([("project", Pipeline([("log", FunctionTransformer(np
                            ("kmeans", KMeans(K, n_init=20, random_state=RANDOM_
final_customers["cluster"] = final_segmenter.predict(final_customers[SEGMENT_FEA

# Align final clusters with the validated discovery segments (same names S1..Sk)
disc_c = segmenter.named_steps["kmeans"].cluster_centers_
fin_c = final_segmenter.named_steps["kmeans"].cluster_centers_
from scipy.optimize import linear_sum_assignment
cost = ((fin_c[:, None, :] - disc_c[None, :, :]) ** 2).sum(-1)
rows, cols = linear_sum_assignment(cost)
final_map = {int(r): SEGMENT_OF[int(c)] for r, c in zip(rows, cols)}
final_customers["segment"] = final_customers["cluster"].map(final_map)

full_support = item_support(df)
fb = basket_matrix(df, full_support[full_support >= MIN_SUPPORT].index)
fi = fpgrowth(fb, min_support=MIN_SUPPORT, use_colnames=True, max_len=3)
try:
    final_rules = association_rules(fi, num_itemsets=len(fb), metric="confidence
except TypeError:
    final_rules = association_rules(fi, metric="confidence", min_threshold=MIN_C
final_rules = final_rules[final_rules["lift"] >= MIN_LIFT]
final_rules = final_rules.assign(antecedent=final_rules["antecedents"].apply(lam
                                 consequent=final_rules["consequents"].apply(lam
validated = {(" + ".join(r["antecedents"]), " + ".join(r["consequents"])) for _,
final_rules["validated_next_year"] = [(a, c) in validated for a, c in zip(final_

out_rules = final_rules[["antecedent", "consequent", "support", "confidence", "l
out_rules.to_csv(APP / "rules.csv", index=False)
final_customers.reset_index().to_csv(APP / "customer_segments.csv", index=False)
joblib.dump(final_segmenter, APP / "segmenter.joblib")
meta = {"created": datetime.now().isoformat(timespec="seconds"), "k": K, "segmen
        "snapshot": str(snapshot.date()), "segments": SEGMENT_ACTIONS, "cluster_
        "profiles": prof.drop(columns="revenue").reset_index().to_dict(orient="r
        "versions": {"python": platform.python_version(), "numpy": np.__version_
(APP / "metadata.json").write_text(json.dumps(meta, indent=2, default=str))
print(f"{len(final_customers):,} customers segmented | {len(out_rules):,} rules
display(final_customers["segment"].value_counts().sort_index().to_frame("custome
```

```
5,830 customers segmented | 623 rules (396 validated) | snapshot 2011-12-10
```

	customers
segment	
S1	1151
S2	1467
S3	1237
S4	1975

In [21]:
```
APP_CODE = r"""
import json
from pathlib import Path
import joblib
import pandas as pd
import streamlit as st

HERE = Path(__file__).parent
st.set_page_config(page_title="Retail insights", page_icon="🛍️")

@st.cache_resource
def load():
    meta = json.loads((HERE / "metadata.json").read_text())
    rules = pd.read_csv(HERE / "rules.csv")
    customers = pd.read_csv(HERE / "customer_segments.csv", dtype={"Customer ID"
    return meta, rules, customers, joblib.load(HERE / "segmenter.joblib")

meta, rules, customers, segmenter = load()
st.title("🛍️ Customer segments & product recommendations")
st.caption(f"Online Retail II · {len(customers):,} customers · {len(rules):,} as
seg_info = meta["segments"]

recs, lookup, new_cust, overview = st.tabs(["Product recommendations", "Customer
with recs:
    products = sorted(set(" + ".join(rules["antecedent"]).split(" + ")))
    product = st.selectbox("Customer has this product in the basket", products)
    only_valid = st.checkbox("Only rules that met the full thresholds again in t
    hits = rules[rules["antecedent"].apply(lambda a: product in a.split(" + "))]
    if only_valid:
        hits = hits[hits["validated_next_year"]]
    if hits.empty:
        st.info("No strong associations for this product.")
    else:
        st.dataframe(hits[["antecedent", "consequent", "confidence", "lift"]].he
        top = hits.iloc[0]
        st.success(f"Suggest **{top['consequent']}** — bought in {top['confidenc
                   f"{top['antecedent']} ({top['lift']:.1f}x more often than ave
with lookup:
    cid = st.text_input("Customer ID", value=customers["Customer ID"].iloc[0])
    row = customers[customers["Customer ID"] == cid.strip()]
    if row.empty:
        st.warning("Customer not found.")
    else:
        r = row.iloc[0]; s = seg_info[r["segment"]]
        st.subheader(s["name"]); st.write(f"**Recommended action:** {s['action']
        c1, c2, c3 = st.columns(3)
        c1.metric("Days since last order", int(r["recency_days"])); c2.metric("O
with new_cust:
```

```
    c1, c2, c3 = st.columns(3)
    rec = c1.number_input("Days since last order", 0, 1000, 30)
    freq = c2.number_input("Number of orders", 1, 500, 3)
    mon = c3.number_input("Total spend (GBP)", 1.0, 1e6, 500.0)
    cluster = int(segmenter.predict(pd.DataFrame([{"recency_days": rec, "frequen
    seg = meta["cluster_to_segment"][str(cluster)]
    st.subheader(seg_info[seg]["name"]); st.write(f"**Recommended action:** {seg
with overview:
    st.dataframe(pd.DataFrame(meta["profiles"]), hide_index=True)
"""
(APP / "app.py").write_text(APP_CODE.strip() + "\n")
v = meta["versions"]
(APP / "requirements.txt").write_text(f"streamlit\nscikit-learn=={v['scikit-lear
compile(APP_CODE, "app.py", "exec")
print("App files:", sorted((p.name, f"{p.stat().st_size / 1e6:.1f} MB") for p in
print(f"When deploying, choose Python {v['python'][:4]} so the pinned versions i
```

```
App files: [('app.py', '0.0 MB'), ('customer_segments.csv', '0.4 MB'), ('metadat
a.json', '0.0 MB'), ('requirements.txt', '0.0 MB'), ('rules.csv', '0.1 MB'), ('se
gmenter.joblib', '0.0 MB')]
When deploying, choose Python 3.13 so the pinned versions install and the segment
er loads.
```

Deploy (free): put `artifacts/app/` in your GitHub repo → share.streamlit.io → *Create app* → `.../app/app.py`; pick the Python version printed above. To change segment names or actions, edit `SEGMENT_ACTIONS` and re-run.

Consolidating the knowledge

- **Agreement with prior knowledge:** the segments match classic RFM intuition — a small group of recent, frequent buyers carries most revenue (17 % of customers → 64 % of revenue) — which builds trust in the result.
- **What is new:** design collections cross product types. For a giftware merchandiser this suggests organising the web shop by *collection* (all "red retrospot" items together) as well as by product type, and bundling matching items.
- **Conflicts to watch:** the strongest "rules" are colour variants; promoting them only helps if the product page doesn't already show all colours.
- **Maintenance:** sales are highly seasonal and the range changes each year (only 717 of 4,047 products reach 1 % support), so segments and rules should be **re-mined quarterly**, and rules re-validated before Q4.

🔍 Critic review — Acting on the knowledge

Paste this stage (markdown + key outputs/figures) into Claude with the prompt below. Do **2–3 rounds**, revising in between, and save the transcripts as `critiques/round_2.md` and `critiques/round_3.md`.

> *You are a world-renowned authority on the Knowledge Discovery in Databases (KDD) process as defined by Fayyad, Piatetsky-Shapiro and Smyth, a sought-after keynote speaker, and the author of several award-winning books on KDD and data mining. Review the **Acting on the knowledge** stage of my retail KDD project below. Check it against*

> *everything this KDD stage requires. Focus especially on: whether the delivered outputs serve the end users identified in Stage 0, whether only validated patterns are promoted, and whether a refresh/maintenance plan exists. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Segment actions were placeholders; no refresh plan.	Named segments with concrete actions; quarterly re-mining recommended.
2	`<fresh-chat critique>`	`<changes>`
3		

Submission checklist

- [x] Analysis written for every stage and `SEGMENT_ACTIONS` filled in (author/course/URLs still to add in the header)
- [x] Critic round 1 done for every stage (`critiques/round_1.md`) — **rounds 2–3 to run in a fresh Claude chat**
- [x] *Run all* completes without errors
- [x] `artifacts/` committed — `artifacts/data/` ignored
- [ ] App deployed; URL in the header and README
- [ ] Medium article published + YouTube walkthrough linked